

The `sum()` function expects one iterable argument, so passing it multiple arguments results in an exception:

```
>>> sum(2, 1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: sum() takes at most 2 arguments (4 given)
```

Meanwhile, the built-in `min()` and `max()` functions, which find the minimum or maximum value of several values, accept a single iterable argument or multiple separate arguments:

```
>>> min([2, 1, 3, 5, 8])
1
>>> min(2, 1, 3, 5, 8)
1
>>> max([2, 1, 3, 5, 8])
8
>>> max(2, 1, 3, 5, 8)
8
```

All of these functions take a varying number of arguments, so why are their parameters designed differently? And when should we design functions to take a single iterable argument or multiple separate arguments using the `*` syntax?

How we design our parameters depends on how we predict a programmer will use our code. The `print()` function takes multiple arguments because programmers more often pass a series of strings, or variables that contain strings, to it, as in `print('My name is', name)`. It isn't as common to collect these strings into a list over several steps and then pass the list to `print()`. Also, if you passed a list to `print()`, the function would print that list value in its entirety, so you can't use it to print the individual values in the list.

There's no reason to call `sum()` with separate arguments because Python already uses the `+` operator for that. Because you can write code like `2 + 4 + 8`, you don't need to be able to write code like `sum(2, 4, 8)`. It makes sense that you must pass the varying number of arguments only as a list to `sum()`.

The `min()` and `max()` functions allow both styles. If the programmer passes one argument, the function assumes it's a list or tuple of values to inspect. If